

Object Oriented Programming using Python



Introduction to Object-Oriented Programming in Python

- Object Oriented Programming is a way of computer programming using the idea of objects ” to represents data and methods.
- It is also, an approach used for creating neat and reusable code instead of a redundant one.

Object-Oriented vs Procedural

Object-Oriented Programming (OOP)	Procedural-Oriented Programming (POP)
Bottom-up approach	Top-down approach
Program is divided into objects/classes	Program is divided into functions
Make use of access modifiers like 'public', 'private' and 'protected'	Doesn't use access modifiers
It is more secure	It is less secure
Object can move freely within member functions	Data can move freely from function to function within programs
It supports inheritance	It does not support inheritance

What are classes and objects?

- A **class** is a collection of objects or you can it is a blueprint of objects defining the common attributes and behavior. Now the question arises, how do you do that?
- The keyword is **class**.

```
In [1]: 1 class class1():  
        2     # This is just a sample for class  
        3     pass
```

Creating an Object and Class in Python

```
In [2]: 1 class employee():
        2     def __init__(self, name, age, emp_id, salary):
        3         self.name = name
        4         self.age = age
        5         self.salary = salary
        6         self.id = emp_id
```

```
In [3]: 1 emp1 = employee('Roman', 22, '0001', 1234)
        2 emp2 = employee('Richard', 23, '0002', 2345)
```

```
In [4]: 1 print(emp1.__dict__)
```

```
{'name': 'Roman', 'age': 22, 'salary': 1234, 'id': '0001'}
```

Important Methodologies

Inheritance

Polymorphism

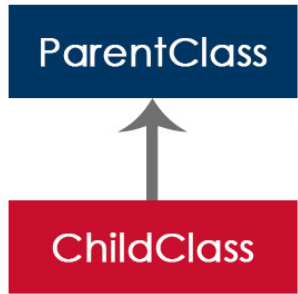
Encapsulation

Inheritance

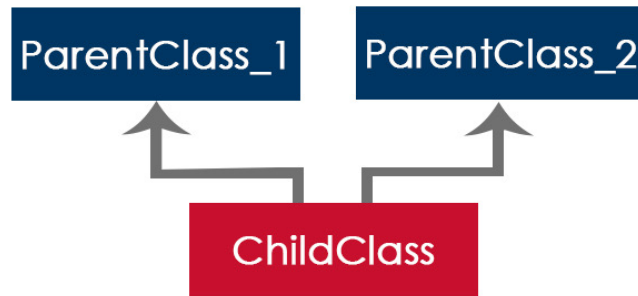
- Ever heard of this dialogue from relatives “you look exactly like your father/mother” the reason behind this is called **inheritance**.
- From the Programming aspect, it generally means “*inheriting or transfer of characteristics from parent to child class without any modification*”
- The new class is called the **derived/child class** and the one from which it is derived is called a **parent/base class**.

Types of Inheritance

Simple Inheritance



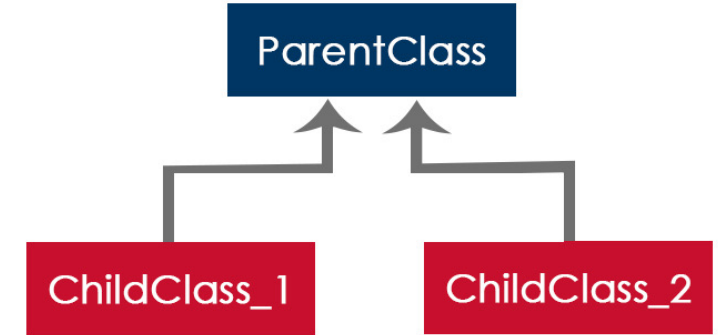
Multiple Inheritance



Multi Level Inheritance



Hierarchical Inheritance



Simple Inheritance

Simple (or single level) inheritance enables a derived class to inherit characteristics from a single parent class

```
1  # Single Inheritance
2  class employee(): # Parent Class
3      emp_id = 0
4
5      def __init__(self, name, age, salary):
6          self.name = name
7          self.age = age
8          temp_id = employee.emp_id
9          employee.emp_id += 1
10         self.id = temp_id
11
12     def printDetails(self):
13         print(self.name, self.id)
```

What are classes and objects?

```
15 class partTime(employee): # Child Class
16
17     def status_PT(self):
18         self.printDetails()
19         print("PART TIME EMPLOYEE")
```

```
21 emp1 = partTime('Roman', 22, 1234.00)
22 emp2 = partTime('Richard', 23, 2345.00)
23
24 emp1.status_PT()
25 emp2.status_PT()
```

```
Roman 0
PART TIME EMPLOYEE
Richard 1
PART TIME EMPLOYEE
```

Multiple Inheritance

Multiple level inheritance enables one derived class to inherit properties from more than one base class.

```
# Multiple Inheritance
class employee(): # Parent Class
    emp_id = 0

    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        temp_id = employee.emp_id
        employee.emp_id += 1
        self.id = temp_id

    def printDetails(self):
        print(self.name, self.id)
```

```
class professional():
    prc_id = 0

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.pro_id = self.getID()

    def getID(self):
        temp_id = professional.prc_id
        professional.prc_id += 1
        self.pro_id = temp_id
        return str(self.pro_id)
```

```
class consultant(employee, professional):

    def status(self):
        print(self.name, self.age)
        print("PROFESSIONAL ID: {}".format(self.getID()))
        print("EMPLOYEE ID: {}".format(self.id), '\n')
```

```
consultant1 = consultant('Roman', 23, 1234.00)
consultant1.printDetails()
consultant1.status()

consultant2 = consultant('Richard', 29, 2345.00)
consultant2.printDetails()
consultant2.status()
```

Multilevel Inheritance (1)

Multi level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class

```
1  # Multilevel Inheritance
2  class employee(): # Parent Class
3      emp_id = 0
4
5      def __init__(self, name, age, salary):
6          self.name = name
7          self.age = age
8          self.salary = salary
9          self.id = setEmpID()
10
11     def printDetails(self):
12         print(self.name, self.id)
13
14     def setEmpID(self):
15         temp_id = employee.emp_id
16         employee.emp_id += 1
17         return temp_id
```

Multilevel Inheritance (2)

Multi level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class

```
19 class middle(employee): # First Derived Class
20
21     def __init__(self, name, age, salary):
22         self.name = name
23         self.age = age
24         self.salary = salary
25         self.id = self.setEmpID()
26         self.deptID = self.setDept()
27
28     def setDept(self, newID=None):
29         if newID == None:
30             print("No Valid Dept ID Set.")
31             self.deptID = int(input("Input new ID: "))
32         else:
33             self.deptID = newID
34             print("Department ID Set.\n")
35
36     def status(self):
37         print("{} has ID No. {}".format(
38             self.name, self.id))
```

Multilevel Inheritance (3)

Multi level inheritance enables a derived class to inherit properties from an immediate parent class which in turn inherits properties from his parent class

```
41 class supervisor(middle):
42
43     def supervise(self):
44         print("Employee {} is now supervising.".format(self.id))
45
46
47 supervisor1 = supervisor('Roman', 29, 1234.00)
48 supervisor1.supervise()
```

No Valid Dept ID Set.

Input new ID: 2

Department ID Set.

Employee 0 is now supervising.

Hierarchical Inheritance (1)

Hierarchical level inheritance enables more than one derived class to inherit properties from a parent class.

```
1  # Hierarchical Inheritance
2  class employee(): # Parent Class
3      emp_id = 0
4
5      def __init__(self, name, age, salary):
6          self.name = name
7          self.age = age
8          self.salary = salary
9          self.id = setEmpID()
10
11     def printDetails(self):
12         print(self.name, self.id)
13
14     def setEmpID(self):
15         temp_id = employee.emp_id
16         employee.emp_id += 1
17         return temp_id
```

Hierarchical Inheritance (2)

Hierarchical level inheritance enables more than one derived class to inherit properties from a parent class.

```
19 class middle(employee): # First Child Class
20
21     def __init__(self, name, age, salary):
22         self.name = name
23         self.age = age
24         self.salary = salary
25         self.id = self.setEmpID()
26         self.deptID = self.setDept()
27
28     def setDept(self, newID=None):
29         if newID == None:
30             print("No Valid Dept ID Set.")
31             self.deptID = int(input("Input new ID: "))
32         else:
33             self.deptID = newID
34             print("Department ID Set.\n")
35
36     def status(self):
37         print("Middle: {} has ID No. {}".format(
38             self.name, self.id))
```

```
40 class top(employee): # Second Child Class
41
42     def __init__(self, name, age, salary):
43         self.name = name
44         self.age = age
45         self.salary = salary
46         self.id = self.setEmpID()
47         self.deptID = self.setDept()
48
49     def setDept(self, newID=None):
50         if newID == None:
51             print("No Valid Dept ID Set.")
52             self.deptID = int(input("Input new ID: "))
53         else:
54             self.deptID = newID
55             print("Department ID Set.\n")
56
57     def status(self):
58         print("Top: {} has ID No. {}".format(
59             self.name, self.id))
```

```
61 emp1 = middle('Roman', 29, 1234.00)
62 emp2 = top('Richard', 29, 2345.00)
63
64 emp1.status()
65 emp2.status()
```

Polymorphism

- An OOP methodology where one task can be performed in several different ways.
- To put it in simple words, it is a property of an object which allows it to take multiple forms.


```

1 # Polymorphism
2 class employee(): # Parent Class
3     emp_id = 0
4
5     def __init__(self, name, age, salary):
6         self.name = name
7         self.age = age
8         self.salary = salary
9         self.id = setEmpID()
10
11     def printDetails(self):
12         print(self.name, self.id)
13
14     def setEmpID(self):
15         temp_id = employee.emp_id
16         employee.emp_id += 1
17         return temp_id

```

```

19 class middle(employee): # First Child Class
20
21     def __init__(self, name, age, salary):
22         self.name = name
23         self.age = age
24         self.salary = salary
25         self.id = self.setEmpID()
26         self.deptID = self.setDept()
27
28     def printDetails(self):
29         print("MIDDLE:")
30         print(self.name, self.id)
31
32     def setDept(self, newID=None):
33         if newID == None:
34             print("No Valid Dept ID Set.")
35             self.deptID = int(input("Input new ID: "))
36         else:
37             self.deptID = newID
38         print("Department ID Set.\n")
39
40     def status(self):
41         print("Middle: {} has ID No. {}".format(
42             self.name, self.id))

```

```

44 class top(employee): # Second Child Class
45
46     def __init__(self, name, age, salary):
47         self.name = name
48         self.age = age
49         self.salary = salary
50         self.id = self.setEmpID()
51         self.deptID = self.setDept()
52
53     def printDetails(self):
54         print("TOP: ")
55         print(self.name, self.id)
56
57     def setDept(self, newID=None):
58         if newID == None:
59             print("No Valid Dept ID Set.")
60             self.deptID = int(input("Input new ID: "))
61         else:
62             self.deptID = newID
63         print("Department ID Set.\n")
64
65     def status(self):
66         print("Top: {} has ID No. {}".format(
67             self.name, self.id))

```

Encapsulation

- Encapsulation is one of the key features of object-oriented programming. Encapsulation refers to the bundling of attributes and methods inside a single class.
- It prevents outer classes from accessing and changing attributes and methods of a class. This also helps to achieve data hiding.

In Python, we denote private attributes using underscore as the prefix i.e single _ or double __. For example,

Example:

```
44 class top(employee): # Second Child Class
45
46     def __init__(self, name, age, salary):
47         self.name = name
48         self.age = age
49         self.__salary = salary
50         self.id = self.setEmpID()
51         self.deptID = self.setDept()
52
53     def printDetails(self):
54         print("TOP: ")
55         print(self.name, self.id)
56
57     def setDept(self, newID=None):
58         if newID == None:
59             print("No Valid Dept ID Set.")
60             self.deptID = int(input("Input new ID: "))
61         else:
62             self.deptID = newID
63             print("Department ID Set.\n")
64
65     def status(self):
66         print("Top: {} has ID No. {}".format(
67             self.name, self.id))
68
69     def printSalary(self):
70         print(self.__salary)
71
72 emp1 = top('Roman', 29, 1234.00)
73 emp1.printSalary()
74 emp1.salary
```